

Reviewer Pack — Minimal Rank of Algebraic Hodge Decomposition Bounds XOR-AND...

Entry f2c664829946 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 17:14:51 UTC

Minimal Rank of Algebraic Hodge Decomposition Bounds XOR-AND Tree Width

Entry ID: f2c664829946 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 17:14:51 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Complexity Theory: XOR-AND Tree Width

Statement:

[‘For every satisfiable Boolean formula F with m clauses and n variables, let H_F be its associated Hodge decomposition. Then the XOR-AND tree width of F is upper-bounded by a function of the rank of the Hodge decomposition of F : $tw(F) \leq c * r(H_F)$, where $r(H_F)$ is the sum of the ranks of all the sheaves in the Hodge decomposition of F and c is an absolute constant.’, ‘For all instances with property P , property Q holds.’, ‘If there exists a Boolean formula F with m clauses and n variables such that $tw(F) > c * r(H_F)$, then the conjecture is falsified.’]

Rationale (proposer’s reasoning):

[‘The Hodge decomposition provides a deep algebraic-geometric insight into the structure of algebraic varieties, which might reveal hidden complexity-theoretic properties.’, ‘XOR-AND tree width is a measure of the depth of computation required for a given Boolean formula, and understanding its bounds can lead to improved algorithms or lower bounds in complexity theory.’, ‘By relating these two fields, the conjecture aims to expose new structures that could potentially be exploited in algorithm design or in proving circuit lower bounds.’]

Taxonomy category: HODGE_XORAND_TREE_WIDTH (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2f45ae50869a0127

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if no seed produces an XOR-AND tree width greater than a constant c times the sum of the ranks of all sheaves in the Hodge decomposition for a

satisfiable Boolean formula, and falsified if any seed produces an XOR-AND tree width exceeding this threshold.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge decomposition" AND "XOR-AND tree width" - "algebraic geometry" AND "Boolean formula complexity" - "Hodge theory" IN author OR "complexity theory" IN author AND "tree width"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(cols):
        max_row = max(range(i, rows), key=lambda r: abs(matrix[r][i]))
        if matrix[max_row][i] == 0:
            return None
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        for j in range(rows):
            if i != j:
                factor = Fraction(matrix[j][i], matrix[i][i])
                for k in range(i, cols):
                    matrix[j][k] -= factor * matrix[i][k]
    return matrix

def xor_and_tree_width(formula):
    # Placeholder function to compute XOR-AND tree width
    # This is a stub and should be replaced with actual computation
    return random.randint(1, 10)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
```

```

m = random.randint(n // 2, n * 2)
formula = [[random.choice([0, 1]) for _ in range(n)] for _ in range(m)]

H_F = gaussian_elimination(formula)
if H_F is None:
    return {
        "metric_name": "XOR-AND tree width",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "singular_matrix"
    }

tw_F = xor_and_tree_width(formula)
r_H_F = sum(sum(1 for x in row if x != 0) for row in H_F)

c = Fraction(2, 1) # Placeholder constant
metric_value = tw_F <= c * r_H_F

return {
    "metric_name": "XOR-AND tree width",
    "metric_value": tw_F,
    "instances_tested": 1,
    "conjecture_holds": metric_value,
    "counterexample": "" if metric_value else f"tw(F) = {tw_F}, c * r(H_F) = {c * r_H_F}"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{'seed': {seed}, **trial_result}}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        std_dev = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
        support_fraction = len([result for result in results if result["conjecture_holds"]]) / len(results)
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TRIAL: {'seed': 11, **trial_result}
```

```
--STDERR--
```

```

Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_df6221e1.py", line 72, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_df6221e1.py", line 43, in run_trial
    H_F = gaussian_elimination(formula)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_df6221e1.py", line 21, in gaussian_elim
    max_row = max(range(i, rows), key=lambda r: abs(matrix[r][i]))
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: max() iterable argument is empty

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means we cannot determine if the conjecture is supported or falsified based on the pre-registered support and falsification conditions. | next: Re-run the test with a different seed to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12469
2	preregistration	ollama_remote	glm4:latest	0	0	5490
3	novelty	ollama_remote	glm4:latest	0	0	4818
4	novelty	ollama_remote	glm4:latest	0	0	5417
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40635
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9261
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9011
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9680
9	judge	ollama_remote	glm4:latest	0	0	9121

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 105902 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/f2c664829946.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/f2c664829946.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f2c664829946.tar.gz` (if generated)